

Akash Banerjee

Profile



Contact



Core Stack

Languages

C++17/20, C, Python
Fortran-facing compiler work

Compiler infrastructure

LLVM, MLIR, Flang
FIR, LLVM IR
OpenMPIRBuilder

Systems

Linux/UNIX
AMDGPU offload
ROCm-facing runtime interfaces
Host/device data movement

Engineering

Low-level debugging
Generated-code correctness
Memory lifetime
ABI/lowering correctness
Regression testing
CI/CD, Git
GDB, LLDB

Education

M.Tech., CSE
IIT Hyderabad
9.50/10, 2021

B.Tech., CSE
RERF, Kolkata
8.37/10, 2017

Compiler engineer at AMD working on LLVM, MLIR, Flang, OpenMP target offload and AMDGPU code generation. I build correctness-critical compiler infrastructure where language semantics, IR lowering, runtime interfaces, memory lifetime and host/device data movement meet.

My recent work includes upstream LLVM changes for OpenMP data mapping, declare mappers, Fortran descriptor handling, GPU offload correctness, AMDGPU complex math lowering and runtime-facing code generation. I enjoy debugging subtle production compiler/runtime failures, designing maintainable abstractions in large C++ codebases, and turning complex specifications into robust generated code.

Experience

2026–Present
2022–2026

Member of Technical Staff at AMD

United Kingdom

Senior Compiler Engineer at AMD

United Kingdom

- Upstream LLVM contributor working across Flang, MLIR, OpenMP dialect lowering, FIR-to-LLVM conversion, LLVM IR translation, OpenMPIRBuilder and AMDGPU offload.
- Built end-to-end OpenMP target offload infrastructure for Flang/MLIR, including target regions, target data, target enter data, target exit data, map metadata generation and runtime-call lowering.
- Designed and implemented OpenMP data-mapping support for Fortran, including user-defined declare mappers, implicit/default mappers, derived-type component mapping, allocatable mappings and nested component handling.
- Fixed correctness-critical offload bugs involving Fortran descriptors, pointer captures, partial maps, recursive mapper emission, mapper symbol lookup, USM semantics and host/device data lifetime.
- Added diagnostics for unsafe target-data mappings involving local Fortran descriptors, preventing misleading GPU offload code when device-side mappings could outlive host stack descriptors.
- Fixed GPU offload crashes for dynamically-sized private arrays by routing boxed array privates through heap allocation with matching cleanup, avoiding runtime-sized scratch allocations captured across GPU callback boundaries.
- Added AMDGPU complex-type and complex-math lowering support across MLIR, ROCm and Flang, including complex argument/return handling, `complex.powi`, `cpow` conversion and a Flang `ConvertComplexPow` pass.
- Upstreamed changes directly to `llvm-project` main through public review, with regression tests and cross-project coordination across Flang, MLIR, LLVM, OpenMP and ROCm-facing paths.

Recognition includes multiple **AMD Spotlight Awards** and an **AMD Executive Award**.

2021–2022

Graphics Compiler Engineer at Imagination Technologies

Kings Langley, UK

- Developed and debugged C++ code-generation paths in a proprietary graphics compiler backend.
- Worked on generated-code correctness, backend lowering behaviour and performance-sensitive compiler issues in a production graphics driver/compiler stack.
- Investigated low-level compiler failures by reasoning across IR transformations, backend code generation and target-specific execution behaviour.

2019–2021

M.Tech. in Computer Science and Engineering – 9.50/10 CGPA

IIT Hyderabad

Publication

- 2025 **Implementing OpenMP Offload Support in the AMD Next Generation Fortran Compiler** LLVM upstream
SC Workshops '25 / ACM
Co-authored paper describing the architecture and implementation of OpenMP target offload in LLVM Flang, including compiler lowering, runtime integration and AMDGPU offload support. ACM DOI.

Selected Technical Work

- 2023–Present **OpenMP data mapping and declare mappers** LLVM upstream
Designed and implemented major parts of OpenMP offload data mapping for Flang/MLIR, translating Fortran semantics, map clauses and derived-type mappings into runtime-facing metadata and generated code. Work includes explicit mappers, implicit/default mappers, nested derived types, allocatables and map-info finalisation.
- 2026 **Offload correctness, descriptor lifetime and GPU memory handling** LLVM upstream
Fixed correctness issues around Fortran descriptors, target-data lifetime, pointer/null handling and dynamically-sized GPU private arrays. Added diagnostics for unsafe local descriptor mappings and fixed GPU crashes caused by runtime-sized private arrays being captured across callback boundaries.
- 2025–2026 **Mapper semantics and AUTOMAP** LLVM upstream
Implemented and stabilised newer OpenMP mapping semantics including AUTOMAP, recursive mapper emission, partial-map handling, default mapper attachment rules, pointer-capture behaviour and inferred mappings for nested allocatable components. AUTOMAP work included parser support, MLIR lowering and an FIR conversion pass.
- 2025 **AMDGPU complex-type and math lowering** LLVM upstream
Closed AMDGPU lowering gaps for complex-valued Fortran/OpenMP workloads, including complex function arguments and returns, MLIR complex-to-ROCDL conversion, `cpow`, `complex.powi` and a Flang `ConvertComplexPow` pass.
- 2022–2024 **Reusable OpenMP offload code generation** LLVM upstream
Refactored OpenMP offload code generation from Clang-specific paths into reusable OpenMPIRBuilder infrastructure shared by Clang and MLIR-based frontends, covering target-data calls, mapping arrays, descriptors, GPU reductions, privatisation and mapper emission.
- 2021 **Proteus: polymorphic compilation** IIT Hyderabad
Built a compiler-based side-channel mitigation using polymorphic compilation and execution, combining program transformation, low-level execution behaviour and security/performance trade-off analysis.
- 2020 **LLVM branch-probability analysis** EuroLLVM poster
Improved LLVM static branch-probability analysis to provide better profiling information to downstream optimisation passes, reporting speed-ups of up to 1.07x.
- 2019 **Pinaka: loop acceleration** GitHub / SV-COMP
Added loop acceleration to Pinaka, a symbolic-execution based C verifier used in SV-COMP, targeting faster counterexample discovery for loops that simulate polynomial functions.
- 2019 **LLVM2GOTO** GitHub
Built a translator from LLVM IR to CPROVER GOTO programs, connecting LLVM frontends with CBMC-style verification workflows.
- 2019 **SAT solvers** GitHub
Implemented DPLL, CDCL and MaxSAT solvers with common decision heuristics and efficient propagation techniques, including MOMS, lazy data structures and watched literals.